

Final Project Report : Server Redundancy Management System (SRMS)

300222668 Farhan Zariwala

1. Introduction and Consolidated Objectives

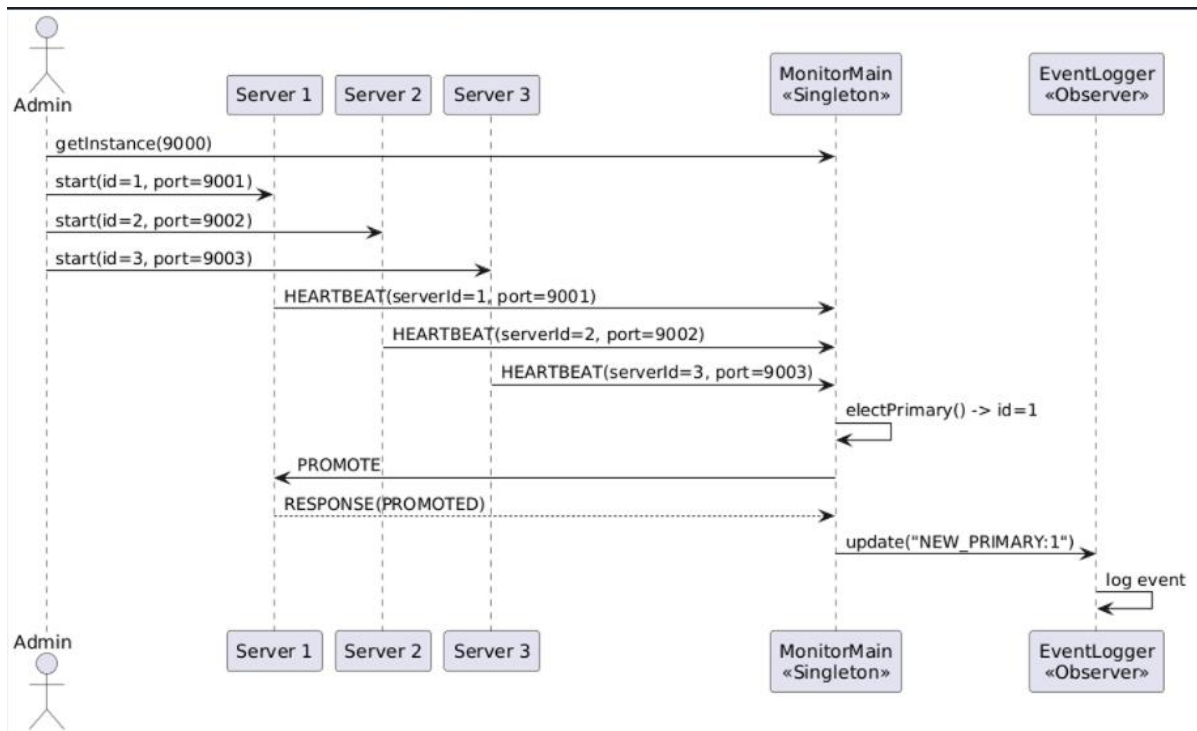
The Server Redundancy Management System (SRMS) was developed over two iterative phases to simulate a highly available, fault-tolerant distributed server cluster. The primary objective of this project was to design a system where a primary server processes client requests while backup servers stand by to automatically take over in the event of a failure.

Rather than treating Mini-Project 1 (Core Implementation) and Mini-Project 2 (Architectural Refactoring) as separate entities, this final iteration integrates both phases. It combines a functional primary-backup failover mechanism with a robust, pattern-based software architecture to ensure both operational reliability and long-term code maintainability.

2. System Architecture and Core Implementation (Phase 1)

The foundational architecture consists of three independent Java server processes, a centralized Monitor, and a Client application.

- **Heartbeat Mechanism:** Servers are configured to send periodic heartbeat messages to the Monitor at 1-second intervals.
- **Client Discovery:** To support dynamic failover, the Client does not hardcode the primary server's IP address. Instead, it queries the Monitor to discover the active primary server before dispatching a PROCESS request.
- **Automated Failover:** The Monitor actively tracks heartbeat timestamps. If a primary server fails to send a heartbeat for 5 consecutive seconds, the Monitor detects the failure, automatically elects a new primary from the available backups (using a deterministic lowest-ID algorithm), and issues a PROMOTE command.



3. Architectural Refactoring and Design Patterns (Phase 2)

To resolve structural limitations identified in the initial build—such as tight coupling, code duplication, and split-brain risks—the system was refactored using three established software design patterns:

3.1 Singleton Pattern

Rationale: The Monitor acts as the central authority for heartbeat tracking and primary election. Only one active instance must exist per system. By implementing the Singleton pattern, we enforce a single, globally accessible instance.

Code Snippet:

```

13     private static MonitorMain instance;

24     private MonitorMain(int port) { this.port = port; }
  
```

```

45     public static synchronized MonitorMain getInstance(int port) {
46         if (instance == null) {
47             instance = new MonitorMain(port);
48         }
49         return instance;
50     }

```

3.2 Abstraction–Occurrence Pattern

Rationale: To avoid duplicating basic server attributes in future extensions, we separated the common fields (the "abstraction") from the concrete execution logic (the "occurrence"). We created an abstract ServerType class to hold the configuration.

Code Snippet:

```

1  package server;
2
3  public abstract class ServerType {
4      protected final int id;
5      protected final int port;
6      protected final String monitorHost;
7      protected final int monitorPort;
8
9      public ServerType(int id, int port, String monitorHost, int monitorPort) {
10         this.id = id;
11         this.port = port;
12         this.monitorHost = monitorHost;
13         this.monitorPort = monitorPort;
14     }
15
16     public abstract void start() throws Exception;
17 }

```

```

public class ServerProcess extends ServerType {
    private volatile boolean running = true;
    private volatile boolean isPrimary = false;
    private final File logFile;

    public ServerProcess (int id, int port, String monitorHost, int monitorPort) {
        super(id, port, monitorHost, monitorPort);
        new File(pathname: "logs").mkdirs();
        this.logFile = new File("logs/server-" + id + ".log");
    }
}

```

3.3 Observer Pattern

Rationale: We needed to decouple the Monitor's core logic from the actions triggered by state changes (e.g., electing a new primary). We introduced an Observer interface, allowing multiple components to "listen" to the Monitor without the Monitor knowing their specific implementations.

Code Snippet:

```

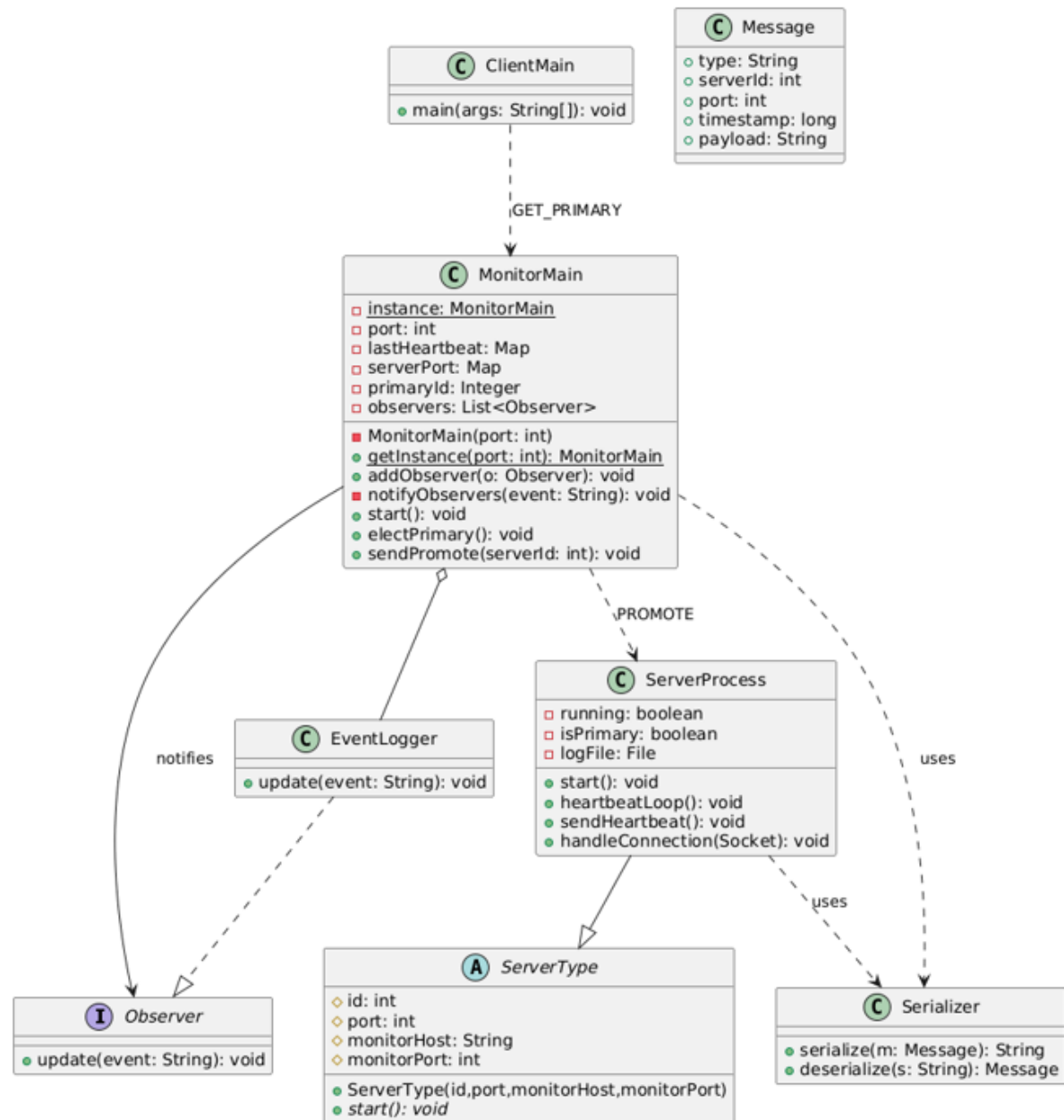
19     private final List<Observer> observers = new ArrayList<>();
20     public void addObserver(Observer o) { observers.add(o); }
21     private void notifyObservers(String event) {
22         for (Observer o : observers) { o.update(event); }
23     }

```

```

110     notifyObservers("NEW_PRIMARY:" + primaryId);

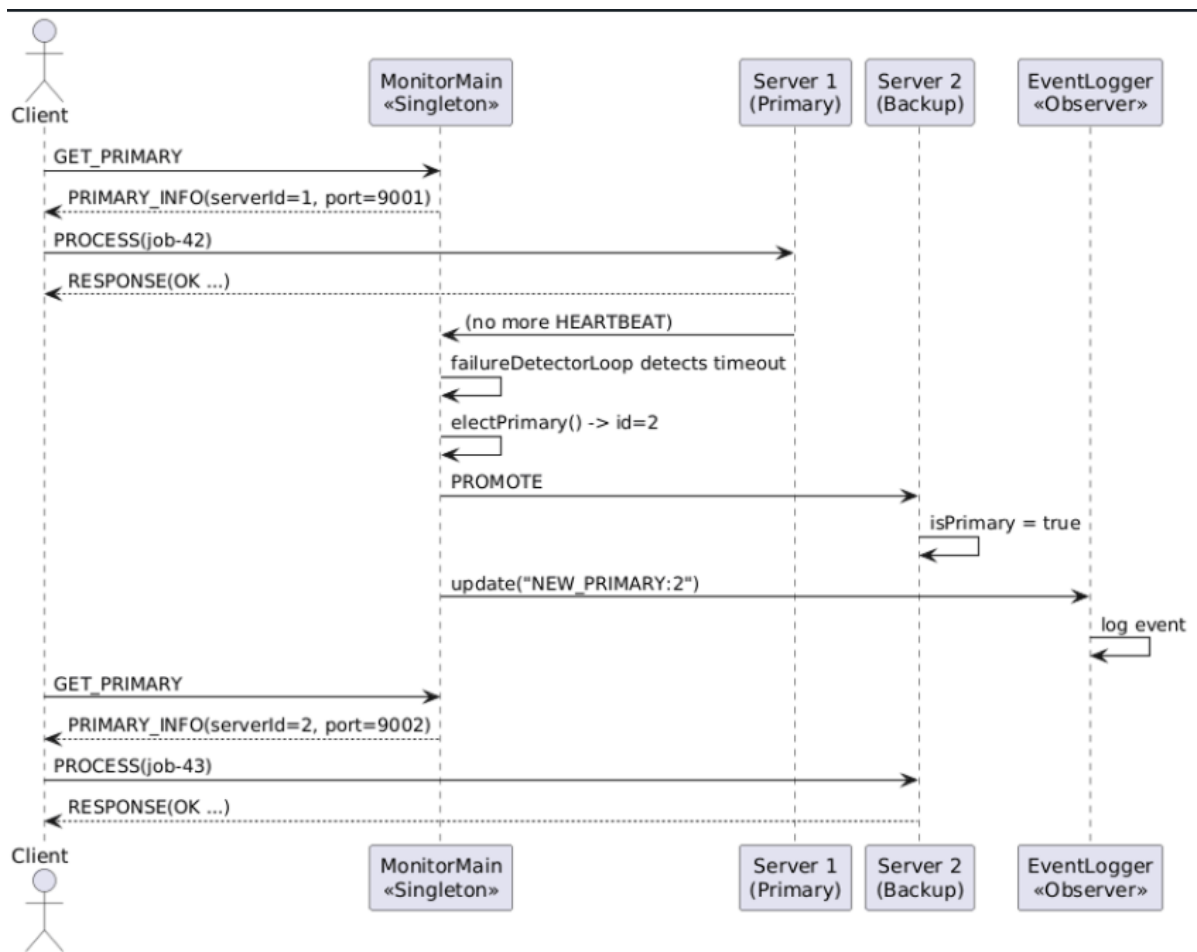
```



4. System Integration and Testing Results

The integration of the core failover logic with the refactored architecture yielded a stable simulation environment. Testing confirmed the following operational results:

- **Successfully Achieved:** The Client successfully queries the Monitor and sends PROCESS requests to the Primary. When the Primary is manually terminated, the Monitor successfully detects the failure via the 5-second timeout and promotes a Backup to Primary. The approximate recovery time from failure detection to service restoration is **~2 seconds**.
- **System Limitations:** While the election and failover mechanisms function flawlessly, **state replication was not successfully achieved**. When a backup server is promoted to primary, it does not currently inherit the memory or transactional history of the previous primary.



5. Challenges Faced

Developing a distributed system introduced several practical software engineering challenges:

1. **Failure Detection Timing:** Distinguishing between a delayed network packet and a crashed server was difficult. Overly aggressive failure detection caused false positives, while lenient detection resulted in unacceptable downtime.
2. **Client Disconnections During Failover:** Initially, when the primary server crashed, the connected Client application threw an `IOException` and terminated entirely, defeating the purpose of a highly available server cluster.
3. **Complexity of State Synchronization:** Attempting to implement continuous state replication across backup nodes introduced severe instability, race conditions, and synchronization bugs.
4. **Refactoring Disruptions:** Applying the Abstraction-Occurrence pattern required moving critical object references into the parent `ServerType` class, which temporarily broke the established failover logic from the first phase of the project.

6. Solutions Tested

To address the aforementioned challenges, the following solutions were engineered and tested:

1. **Empirical Timeout Tuning:** Through iterative testing, the heartbeat and timeout thresholds were explicitly balanced. A 1-second heartbeat paired with a strict 5-second timeout provided the optimal balance between avoiding false alarms and ensuring rapid recovery.
2. **Resilient Client Reconnection Loop:** To prevent client crashes, network calls were wrapped in exception-handling blocks. Upon encountering an `IOException`, the Client catches the error, sleeps briefly, re-queries the Monitor for the new Primary's address, and reconnects seamlessly.
3. **Strategic Scope Reduction:** Rather than delivering an unstable system attempting to do everything, a strategic engineering decision was made to halt state replication efforts. Scope was reduced to guarantee a 100% reliable Primary-to-Backup failover mechanism rather than a partially functioning replication system.
4. **Incremental Pattern Integration:** The system was refactored incrementally. The Singleton pattern was applied and tested first, followed by the Observer pattern, ensuring the core failover logic remained stable throughout the refactoring process.

7. Lessons Learned

The completion of this project provided critical insights into distributed systems engineering:

- **Distributed State is the True Bottleneck:** Electing a new primary server is computationally straightforward; accurately synchronizing data across distributed nodes without causing network latency is substantially more difficult.
- **High Availability Requires Resilient Clients:** Server redundancy is ineffective if the client application is brittle. Fault tolerance must be implemented on both the server side and the client side.
- **Design Patterns Provide Long-Term Value:** While implementing patterns like Observer and Singleton required upfront effort and temporarily disrupted the codebase, the resulting architecture was measurably easier to debug, extend, and maintain.
- **Scope Management is an Engineering Skill:** Delivering a highly reliable core feature (primary failover) is vastly superior to deploying a fully featured but unstable system.

8. Conclusion

The Server Redundancy Management System successfully demonstrates the core principles of distributed fault tolerance. By integrating heartbeat monitoring, automated leader election, and architectural design patterns, the system achieves reliable primary failover with a ~2-second recovery time. Future iterations of this project would focus on implementing Write-Ahead Logging (WAL) to facilitate full state replication and transitioning the system from a local simulation to a true network deployment.

9. Contribution

- **Farhan Zariwala 300222668:** Conducted UML review, implemented Singleton in MonitorMain, applied Abstraction-Occurrence to ServerProcess, implemented Observer pattern, generated updated PlantUML diagrams, and wrote the comparative evaluation report.